You don't need to pass all attributes to `update`. For example, if you'd call `@article.update(title: 'A new title')` Rails would only update the `title` attribute, leaving all other attributes untouched.

Finally, we want to show a link to the `edit` action in the list of all the articles, so let's add that now to `app/views/articles/index.html.erb` to make it appear next to the "Show" link:

```
<table>
  <tr>
    <th>Title</th>
    <th>Text</th>
    <th colspan="2"></th>
  </tr>

  <% @articles.each do |article| %>
    <tr>
      <td><%= article.title %></td>
      <td><%= article.text %></td>
      <td><%= link_to 'Show', article_path(article) %></td>
      <td><%= link_to 'Edit', edit_article_path(article) %></td>
    </tr>
  <% end %>
</table>
```

And we'll also add one to the `app/views/articles/show.html.erb` template as well, so that there's also an "Edit" link on an article's page. Add this at the bottom of the template:

```
...

<%= link_to 'Edit', edit_article_path(@article) %> |
<%= link_to 'Back', articles_path %>
```

And here's how our app looks so far:



5.12 Using partials to clean up duplication in views

Our `edit` page looks very similar to the `new` page; in fact, they both share the same code for displaying the form. Let's remove this duplication by using a view partial. By convention, partial files are prefixed with an underscore.

You can read more about partials in the Layouts and Rendering in Rails guide.

Create a new file `app/views/articles/_form.html.erb` with the following content:

```
<%= form_for @article do |f| %>

  <% if @article.errors.any? %>
    <div id="error_explanation">
      <h2>
        <%= pluralize(@article.errors.count, "error") %> prohibited
        this article from being saved:
      </h2>
      <ul>
        <% @article.errors.full_messages.each do |msg| %>
          <li><%= msg %></li>
        <% end %>
      </ul>
    </div>
  <% end %>
```